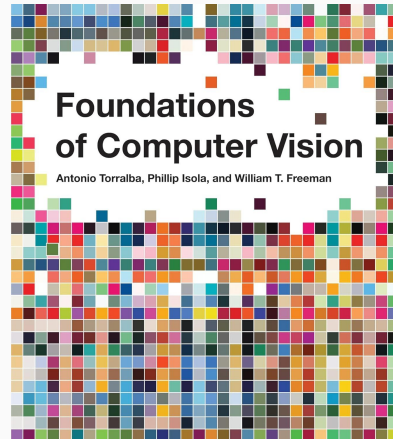


Preparation in Computer Vision



Χρήσιμο υλικό



<https://visionbook.mit.edu/>

Foundation of Computer Vision

Τι είναι η Υπολογιστική Όραση (Computer Vision);

- Η Υπολογιστική Όραση είναι ένας κλάδος της Τεχνητής Νοημοσύνης που επιτρέπει στους υπολογιστές να **κατανοούν, να ερμηνεύουν και να αναλύουν οπτική πληροφορία** από εικόνες και βίντεο.
- Στόχος της είναι να δώσει στις μηχανές τη δυνατότητα να εκτελούν εργασίες που ο άνθρωπος πραγματοποιεί φυσικά μέσω της όρασης, όπως:
 - **Αναγνώριση αντικειμένων και σκηνών**
 - **Εντοπισμός και εντοπισμός θέσης αντικειμένων**
 - **Κατανόηση του περιεχομένου μιας εικόνας**
 - **Δημιουργία και επεξεργασία οπτικών δεδομένων**
- Η σύγχρονη Υπολογιστική Όραση βασίζεται σε:
 - **Μοντέλα βαθιάς μάθησης** (CNNs, Vision Transformers)
 - **Μεγάλα σύνολα δεδομένων**
 - **Αυτο-επιβλεπόμενη και πολυτροπική μάθηση**

Δεκαετία 1960 – Τα πρώτα βήματα της Υπολογιστικής Όρασης

- Το **1966**, στο MIT, προτάθηκε το **Summer Vision Project**, με στόχο την ανάπτυξη ενός συστήματος που θα μπορούσε να αναγνωρίζει αντικείμενα μέσα σε εικόνες. Η ιδέα ήταν ότι η κατανόηση της όρασης θα μπορούσε να λυθεί μέσα σε ένα **καλοκαιρινό ερευνητικό πρόγραμμα**.
- Στην πράξη, όμως, η όραση των υπολογιστών αποδείχθηκε πολύ πιο δύσκολη:
 - Οι εικόνες περιέχουν τεράστια πολυπλοκότητα.
 - Η αναγνώριση αντικειμένων απαιτεί κατανόηση του περιβάλλοντος και της σημασίας των δεδομένων.

Ιστορική Αναδρομή

1. Δεκαετίες 1970–1990: Κλασική Υπολογιστική Όραση

Μετά το Summer Vision Project, η έρευνα στράφηκε στην εξαγωγή χαρακτηριστικών από εικόνες.

- Οι υπολογιστές δεν μπορούσαν ακόμη να «καταλάβουν» εικόνες.
- Οι ερευνητές σχεδίαζαν χειροκίνητα χαρακτηριστικά:
 - ακμές (edges)
 - γωνίες (corners)
 - σχήματα και υφές (textures)
- Στόχος: μετατροπή της εικόνας σε αριθμητική αναπαράσταση που μπορούσαν να επεξεργαστούν οι αλγόριθμοι.

3. 2012: Η επανάσταση του Deep Learning

Το σημείο καμπής ήταν το AlexNet στο ImageNet.

- Τα Convolutional Neural Networks (CNNs) έμαθαν αυτόματα χαρακτηριστικά από δεδομένα.
- Δεν χρειαζόταν πλέον χειροκίνητος σχεδιασμός χαρακτηριστικών.
- Μεγάλα datasets + GPUs → μεγάλη αύξηση της απόδοσης.

2. Δεκαετίες 2000–2010: Machine Learning στην Όραση

Η ιδέα μετακινήθηκε από το "σχεδιάζω χαρακτηριστικά" στο "μαθαίνω χαρακτηριστικά".

- Χρήση αλγορίθμων όπως:
 - SVM
 - Random Forests
- Συνδυασμός με χαρακτηριστικά όπως:
 - SIFT
 - HOG
- Οι υπολογιστές άρχισαν να αναγνωρίζουν αντικείμενα με μεγαλύτερη ακρίβεια.

4. Σήμερα: Foundation Models & Generative AI

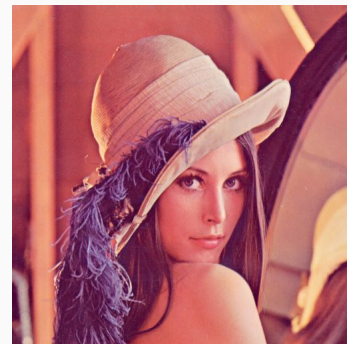
Η Υπολογιστική Όραση πέρασε από την αναγνώριση στην κατανόηση και δημιουργία.

- Vision Transformers (ViT)
- Self-supervised learning (DINO, MAE)
- Vision-Language Models (CLIP)
- Generative models (Stable Diffusion)

Τι είναι εικόνα;

Τι είναι μια εικόνα για έναν υπολογιστή;

- Για τον άνθρωπο, μια εικόνα είναι μια **οπτική πληροφορία** με αντικείμενα, χρώματα και νοήματα.
- Για έναν υπολογιστή, μια εικόνα είναι απλά ένας **πίνακας αριθμών (matrix)**.
- Κάθε εικόνα αποτελείται από μικρά στοιχεία που ονομάζονται **pixels**:
 - Κάθε ρixel περιέχει αριθμητικές τιμές που αναπαριστούν την ένταση ή το χρώμα.
- **Grayscale εικόνα:**
 - Ένας πίνακας $H \times W$
 - Κάθε τιμή δείχνει την ένταση φωτεινότητας (0–255)
- **Έγχρωμη εικόνα (RGB):**
 - Ένας τρισδιάστατος πίνακας $H \times W \times 3$
 - Τα 3 κανάλια αντιστοιχούν σε:
 - Κόκκινο (Red)
 - Πράσινο (Green)
 - Μπλε (Blue)



Η Lena (1972) — μία από τις πιο εμβληματικές εικόνες benchmark στην ιστορία της επεξεργασίας εικόνων

Low-Level Features: Ακμές και Βασικά Χαρακτηριστικά

Low-Level Features

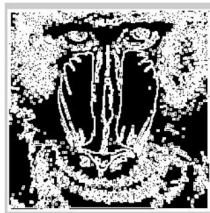
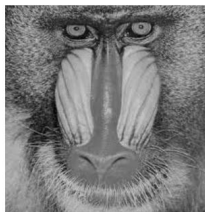
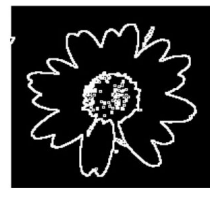
Περιλαμβάνουν:

- **Edges** → όρια και περιγράμματα
- **Corners** → χαρακτηριστικά σημεία (π.χ. γωνίες αντικειμένων)
- **Textures** → επαναλαμβανόμενα μοτίβα
- **Contours** → σχήματα και περιγράμματα

Edges (Ακμές)

Edges (Ακμές)

- Οι ακμές είναι σημεία όπου υπάρχει **απότομη αλλαγή στην ένταση ή στο χρώμα των pixels**.
- Αντιπροσωπεύουν τα **όρια μεταξύ διαφορετικών περιοχών** μιας εικόνας.
- Χρησιμοποιούνται για τον εντοπισμό:
 - Περιγραμμάτων αντικειμένων
 - Αλλαγών φωτισμού
 - Δομής και γεωμετρίας μιας εικόνας
- Παράδειγμα: το όριο μεταξύ ενός αντικειμένου και του φόντου.

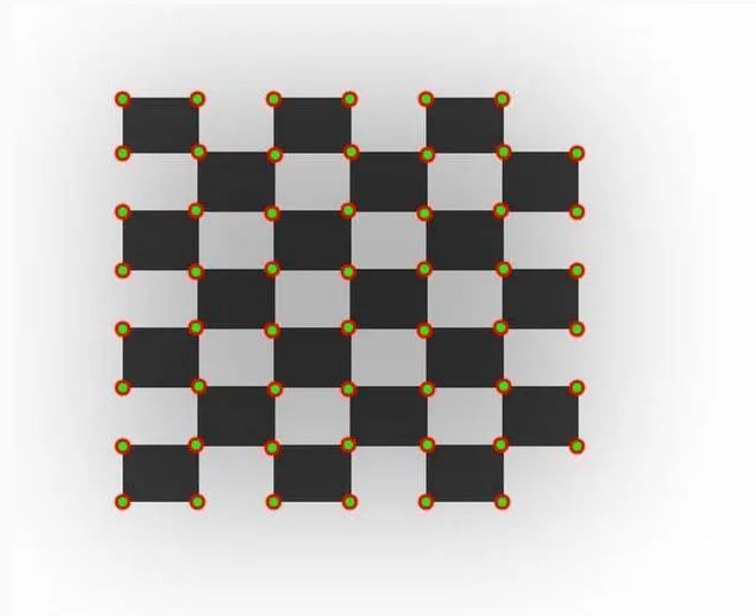


Corners → χαρακτηριστικά σημεία (π.χ. γωνίες αντικειμένων)

Corners (Γωνίες / Χαρακτηριστικά σημεία)

- Είναι σημεία όπου υπάρχουν **έντονες αλλαγές προς πολλές κατευθύνσεις**.
- Αποτελούν μοναδικά χαρακτηριστικά που μπορούν να εντοπιστούν σε διαφορετικές εικόνες.
- Χρησιμοποιούνται για:
 - Αναγνώριση αντικειμένων
 - Αντιστοίχιση σημείων μεταξύ εικόνων (image matching)
 - Υπολογισμό κίνησης σε βίντεο
- Παράδειγμα: γωνίες ενός βιβλίου ή ενός κτιρίου.

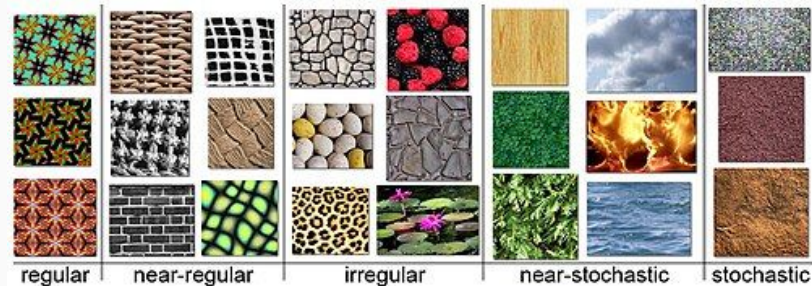
Harris corner detection.



Textures → επαναλαμβανόμενα μοτίβα

Textures (Υφές)

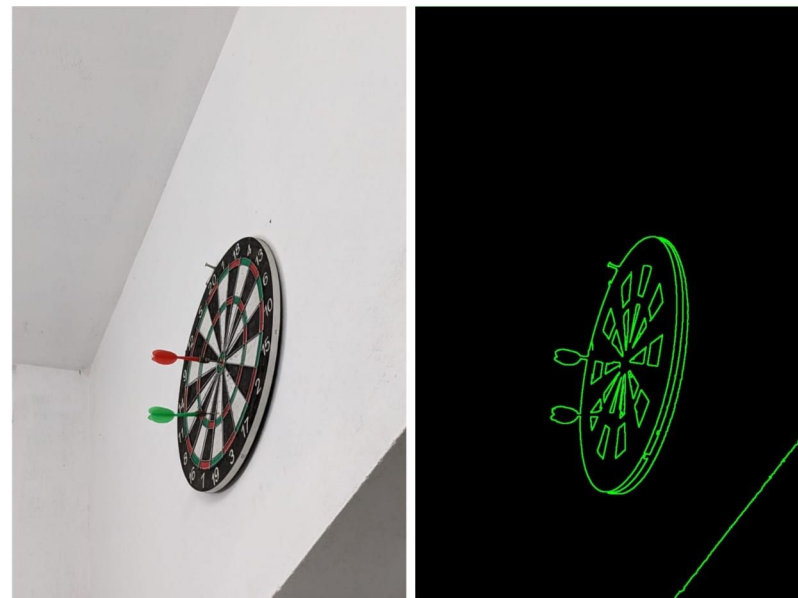
- Περιγράφουν τα **επαναλαμβανόμενα μοτίβα** ή την **τοπική δομή** μιας περιοχής της εικόνας.
- Δεν περιγράφουν συγκεκριμένα αντικείμενα, αλλά την εμφάνιση μιας επιφάνειας.
- Χρησιμοποιούνται για την αναγνώριση:
 - Υλικών
 - Επιφανειών
 - Περιοχών με παρόμοια χαρακτηριστικά



Contours → σχήματα και περιγράμματα

Contours (Περιγράμματα)

- Είναι καμπύλες που περιγράφουν το **σχήμα και τα όρια ενός αντικειμένου**.
- Προκύπτουν συνήθως από την ανίχνευση ακμών.
- Χρησιμοποιούνται για:
 - Ανάλυση σχημάτων
 - Μέτρηση αντικειμένων
 - Αναγνώριση γεωμετρικών χαρακτηριστικών
- Παράδειγμα: το περίγραμμα ενός προσώπου ή ενός κυπέλλου.



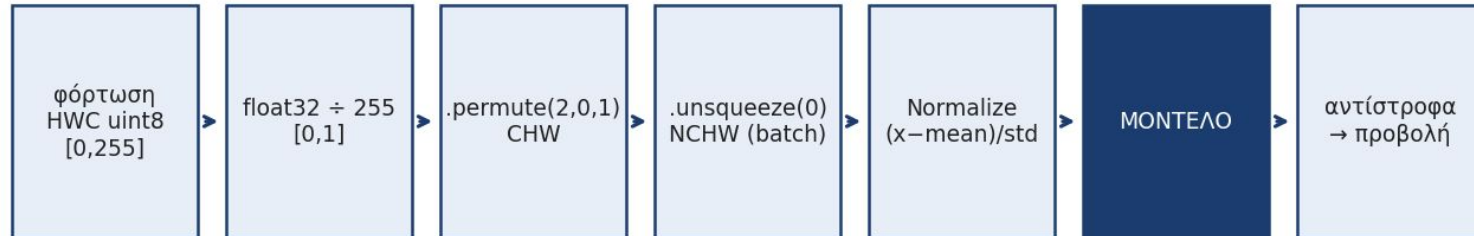
Από Pixels σε Tensors: το pipeline κάθε μοντέλου

Τρία layouts για την ίδια εικόνα

HWC (H,W,C): NumPy, OpenCV, matplotlib · CHW (C,H,W): PyTorch · NCHW: batch N εικόνων.

Λάθος layout $\neq$ error — δίνει λάθος χρώματα ή shape mismatch.

Τιμές: αποθήκευση uint8 [0,255] $\rightarrow$ είσοδος δικτύου float32 [0,1] $\rightarrow$ standardization ανά κανάλι (ImageNet mean/std).

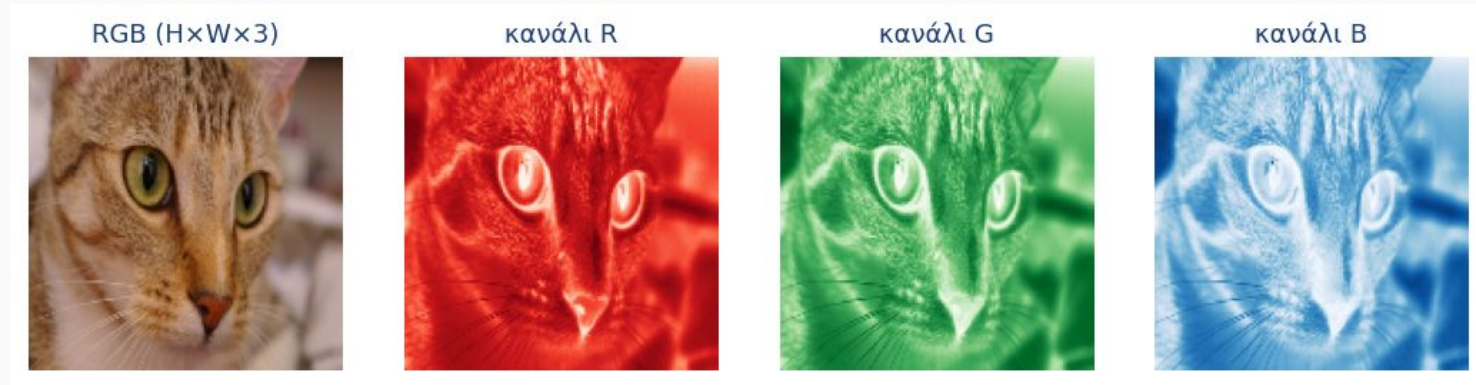


Μια Εικόνα είναι Πίνακας Αριθμών

Grayscale: πίνακας $H \times W$ — μία τιμή έντασης ανά pixel (0 = μαύρο, 255 = λευκό).

Έγχρωμη: τρεις πίνακες στοιβαγμένοι — κανάλια Red, Green, Blue.

Κάθε μοντέλο όρασης δουλεύει πάνω σε αυτόν τον αριθμητικό πίνακα.



Φίλτρα και Συνέλιξη (Filtering)

Συνέλιξη = ολίσθηση kernel

Ένα kernel $k \times k$ περνά από κάθε θέση · έξοδος = σταθμισμένο άθροισμα της γειτονιάς.

Gaussian blur → εξομάλυνση · Sharpen → ενίσχυση ακμών · Sobel → gradient.

Ίδια πράξη με τα conv layers — εκεί τα βάρη μαθαίνονται.

Είσοδος (grayscale)



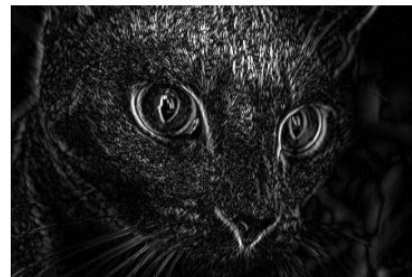
Gaussian blur



Sharpen



Sobel $\partial/\partial x$



Μαθηματικά: Συνέλιξη και Διαστάσεις

$$(\mathbf{I} * \mathbf{K})(x, y) = \sum_i \sum_j \mathbf{I}(x+i, y+j) \mathbf{K}(i, j)$$

Διακριτή 2D συνέλιξη: το kernel K αθροίζει σταθμισμένα τη γειτονιά κάθε pixel.

$$H_{out} = \lfloor (H_{in} + 2p - k)/s \rfloor + 1$$

Διάσταση εξόδου από padding p, kernel k, stride s — προβλέπεις shapes χωρίς να τρέξεις το δίκτυο.

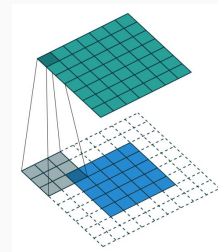
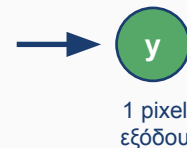
3	0	1	2	7
1	5	8	9	3
2	7	2	5	1
0	1	3	1	7
4	2	1	6	2

3×3 patch εισόδου

kernel K

-1	0	1
-1	0	1
-1	0	1

πολλαπλασιασμός
και άθροισμα



Ανίχνευση Ακμών και Κατωφλίωση

1. Ακμές: Canny (~Sobel) — 4 βήματα

1. Gaussian smoothing → 2. Sobel gradients
3. Non-maximum suppression → 4. Hysteresis (2 κατώφλια).

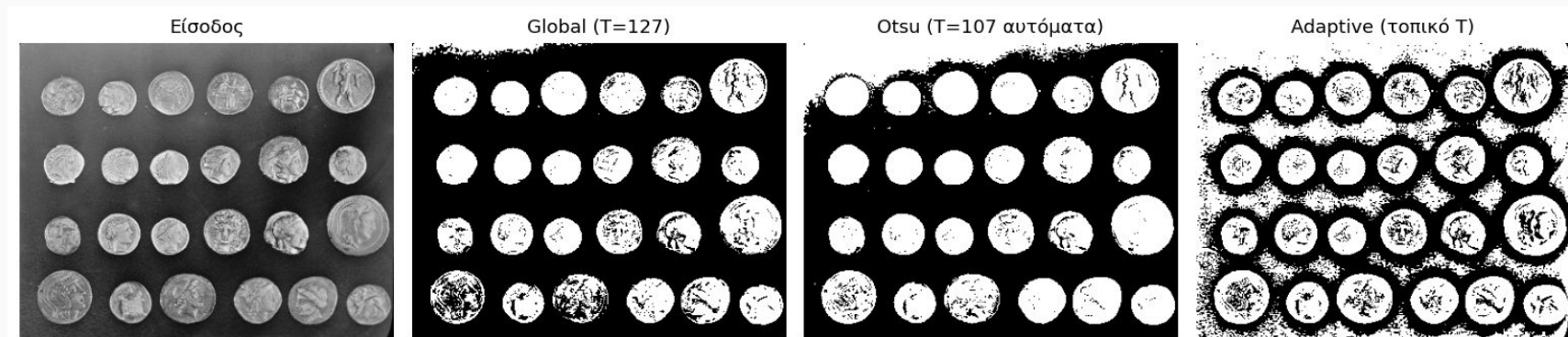
2. Κατωφλίωση:

Thresholding (Απο-ασαφопоίησης) — 3 τρόποι

Global: σταθερό T για όλη την εικόνα. Εγώ ορίζω μια τιμή και απο εκεί και πέρα 0.

Otsu: αυτόματη επιλογή του T . (2 κλάσεις - 0/1) - maximize το variance των κλάσεων

Adaptive: τοπικό T ανά γειτονιά — για ανομοιομόρφο φωτισμό. Παίρνεις περιοχές spatially και apply Otsu ή το Global.



Μαθηματικά: Canny και Otsu

$$G = \sqrt{G_x^2 + G_y^2}, \quad \theta = \arctan(G_y/G_x)$$

Μέτρο και κατεύθυνση gradient ανά pixel. Το NMS κρατά τα τοπικά μέγιστα κατά τη $\theta \rightarrow$ ακμές 1 px.

$$G \geq T_{high} \Rightarrow \text{edge}; \quad T_{low} \leq G < T_{high} \Rightarrow \text{edge if connected}$$

Ισχυρές ακμές πάντα· ασθενείς μόνο αν συνδέονται με ισχυρή $\rightarrow$ συνεχή περιγράμματα.

$$T^* = \arg \max_T \omega_0(T) \omega_1(T) (\mu_0(T) - \mu_1(T))^2$$

Otsu: το T που μεγιστοποιεί τη διακύμανση μεταξύ των δύο κλάσεων (ω = ποσοστά, μ = μέσοι).

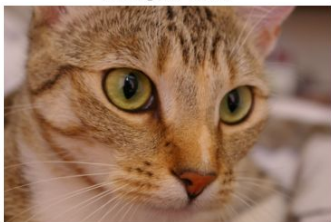
HOG και το Κλασικό Pipeline

HOG σε 4 βήματα

Gradient ανά pixel $\rightarrow$ ιστόγραμμα 9 κατευθύνσεων ανά cell $\rightarrow$ κανονικοποίηση ανά block $\rightarrow$ ένα διάνυσμα.

HOG + linear SVM: η κλασική ανίχνευση πεζών. Καμία μάθηση χαρακτηριστικών — μόνο μαθηματικά στα pixels.

1. Original (RGB)



2. Grayscale



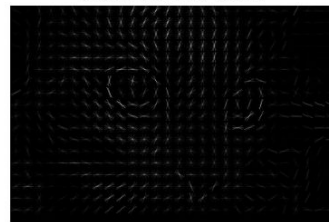
3. Gaussian blur (7×7 , $\sigma=1.5$)



4. Canny (50, 150)



5. HOG (16524-d vector)



Συνελικτικά Δίκτυα (CNN)

Από χειροποίητα σε μαθημένα φίλτρα

Τα βάρη των kernels μαθαίνονται με backpropagation.

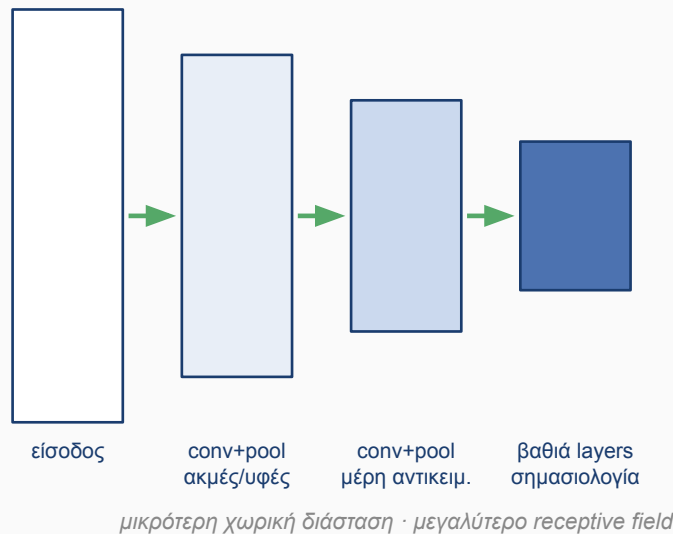
Weight sharing: ίδιο kernel παντού → λίγες παράμετροι, translation equivariance.

Pooling: υποδειγματοληψία → μεγαλύτερο receptive field.

Ιεραρχία

Ακμές → υφές → μέρη αντικειμένων → έννοιες.

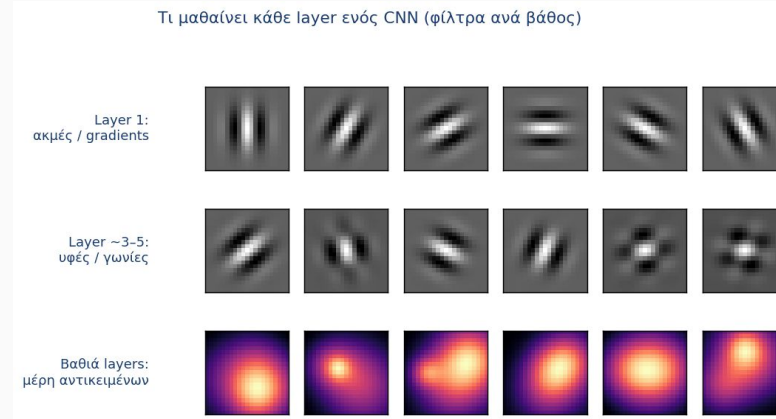
Βάθος → πιο αφηρημένα χαρακτηριστικά



Τι Μαθαίνει Κάθε Layer

Τα φίλτρα προκύπτουν από την εκπαίδευση — δεν σχεδιάζονται.

Layer 1: ανιχνευτές ακμών, παρόμοιοι με Gabor/Sobel · μεσαία: υφές, γωνίες · βαθιά: μέρη αντικειμένων ανά κλάση.

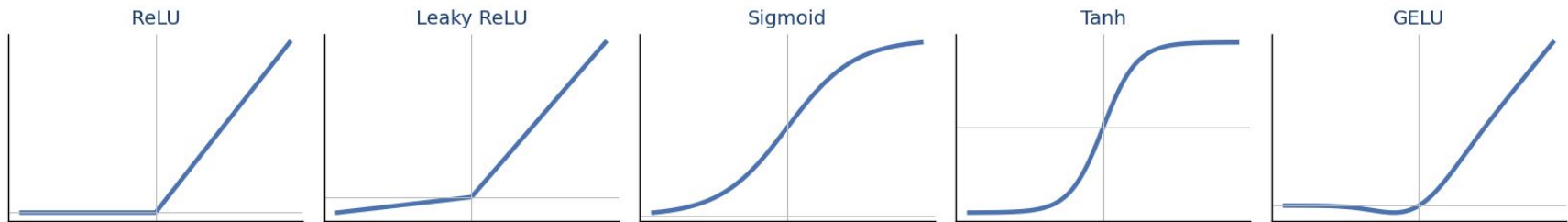


Συναρτήσεις Ενεργοποίησης

Χωρίς μη γραμμικότητα, στοιβαγμένα linear layers καταρρέουν σε ένα.

ReLU: η προεπιλογή στα CNN · Leaky ReLU: αποφεύγει «νεκρά» νευρώνες · Sigmoid/Tanh: κορεσμός στα άκρα · GELU: ομαλή — η επιλογή των transformers.

Συναρτήσεις ενεργοποίησης: εισάγουν μη γραμμικότητα



$$\text{ReLU}(x) = \max(0, x)$$

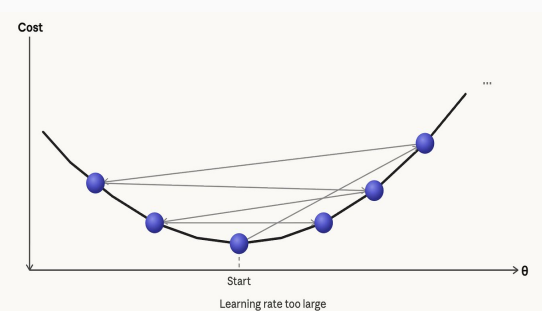
Η ReLU μηδενίζει τα αρνητικά — απλή, γρήγορη μη γραμμικότητα.

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

Gradient descent: βήμα αντίθετα στην κλίση, με ρυθμό μάθησης η .

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \prod_{k=l+1}^L \frac{\partial \mathbf{a}^{(k)}}{\partial \mathbf{a}^{(k-1)}} \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{W}^{(l)}}$$

Backpropagation = κανόνας αλυσίδας: γινόμενο των τοπικών κλίσεων όλων των επόμενων layers.



Normalization Layers

Γιατί

Σταθερή κατανομή ενεργοποιήσεων $\rightarrow$ ταχύτερη, σταθερότερη εκπαίδευση.

Batch Norm

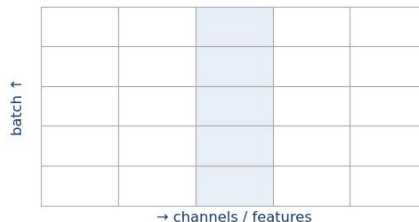
Στατιστικά ανά κανάλι, σε όλο το batch · μαθαίνει γ , β .
Ευαίσθητο σε μικρό batch.

Layer Norm

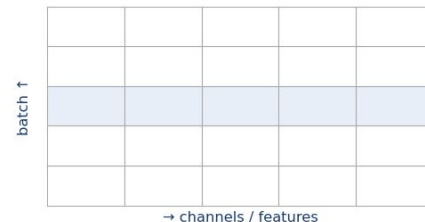
Στατιστικά ανά δείγμα — ανεξάρτητο από batch · η επιλογή των transformers.

Normalization: σταθεροποιεί την κατανομή των ενεργοποιήσεων

Batch Norm
(ανά κανάλι, σε όλο το batch)



Layer Norm
(ανά δείγμα, σε όλα τα features)



Μαθηματικά: Batch Normalization

Στο inference: running mean/var αντί για στατιστικά batch.

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

Αφαίρεση μέσου μ_B , διαίρεση με τυπική απόκλιση του batch· μετά μαθημένα γ , β επαναφέρουν εκφραστικότητα.

$$\text{GELU}(x) = x \Phi(x)$$

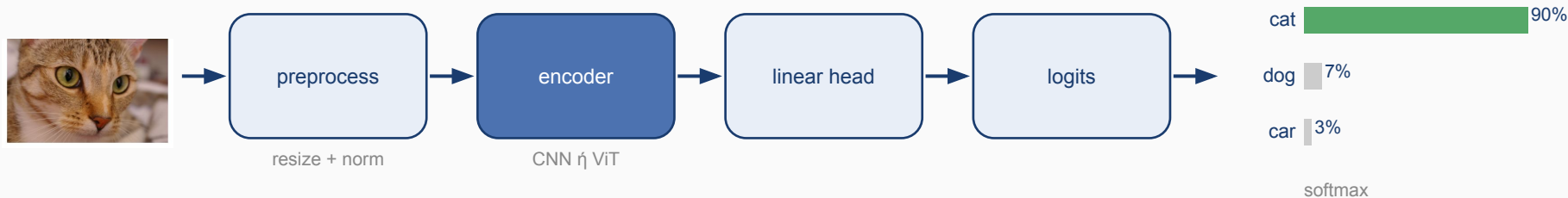
GELU: x επί την αθροιστική κανονική $\Phi(x)$ — ομαλή εναλλακτική της ReLU.

Ταξινόμηση Εικόνας (Image Classification)

Task: εικόνα $\rightarrow$ μία ετικέτα από σταθερό σύνολο κλάσεων.

Ροή: preprocess $\rightarrow$ encoder $\rightarrow$ linear head $\rightarrow$ logits $\rightarrow$ softmax $\rightarrow$ πιθανότητες.

Loss: cross-entropy · Μετρικές: top-1 / top-5 accuracy.



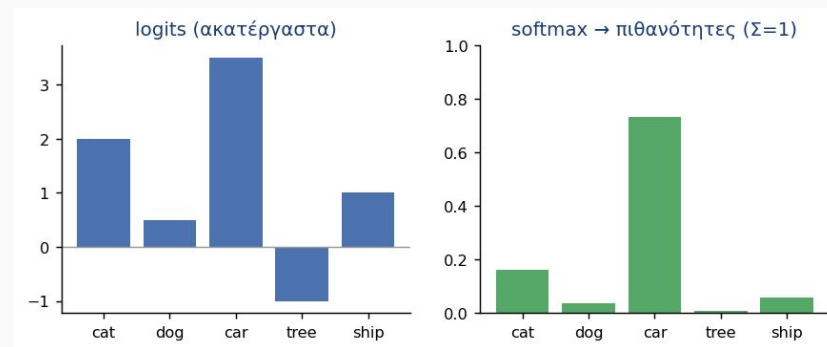
Μαθηματικά: Softmax και Cross-Entropy

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Logits → πιθανότητες: θετικές, άθροισμα 1.

$$\mathcal{L}_{CE} = - \sum_{i=1}^K y_i \log \hat{p}_i = -\log \hat{p}_c$$

Cross-entropy = $-\log$ πιθανότητας της σωστής κλάσης· ελαχιστοποιείται όταν αυτή $\rightarrow 1$.



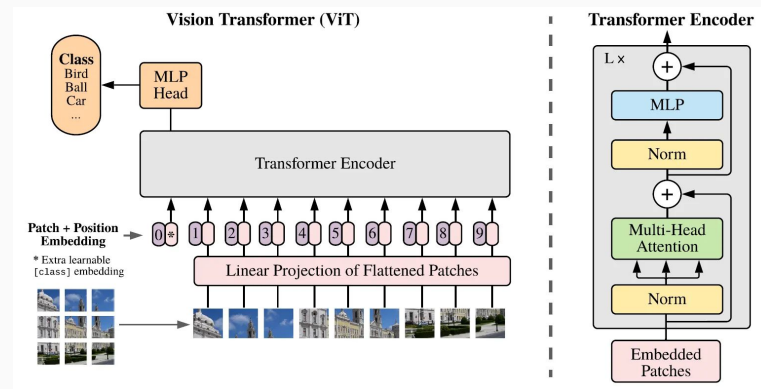
```
logits = model(x)      # OXI softmax  
loss = nn.CrossEntropyLoss()(logits, labels)
```

Το παραπάνω ισοδυναμεί:

```
log_probs = nn.LogSoftmax(dim=1)(logits)  
loss = nn.NLLLoss()(log_probs, labels)
```

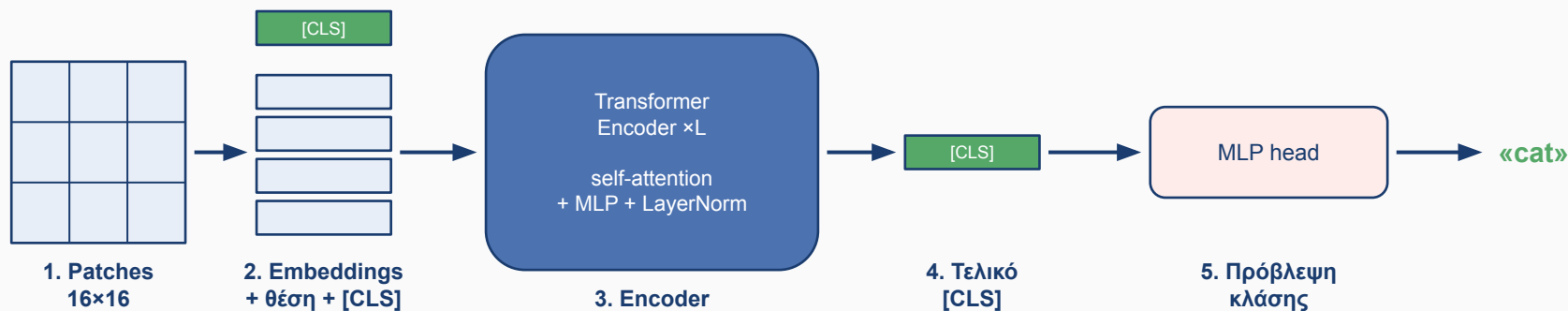
Vision Transformer (ViT)

- **Η ιδέα**
- Η εικόνα γίνεται ακολουθία: patches $16 \times 16 \rightarrow$ tokens.
- Positional embeddings δίνουν τη θέση – το attention αγνοεί τη σειρά.
- **Γιατί διαφέρει από CNN**
- Ολικό receptive field από το 1ο layer.
- Χωρίς inductive biases $\rightarrow$ χρειάζεται μεγάλα datasets ή self-supervision.
- Το [CLS] token συνοψίζει την εικόνα.



ViT: Αρχιτεκτονική Βήμα-Βήμα

1. Patches → 2. embeddings + θέση + [CLS] → 3. Transformer encoder $\times L$ (attention + MLP + LayerNorm) → 4. τελικό [CLS] → 5. MLP head → κλάση.



Μαθηματικά: Self-Attention (Transformer)

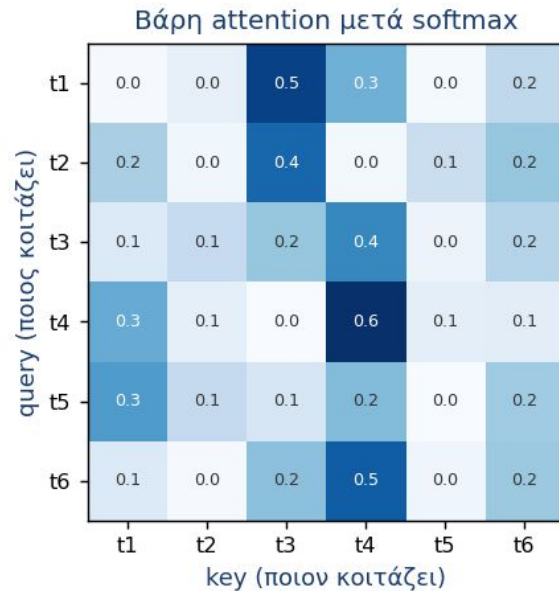
Κόστος $O(n^2)$ στα n tokens — γι' αυτό μετράει η ανάλυση των patches.

$$\mathbf{Q} = \mathbf{XW}_Q, \quad \mathbf{K} = \mathbf{XW}_K, \quad \mathbf{V} = \mathbf{XW}_V$$

Κάθε token προβάλλεται σε queries, keys, values.

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\mathbf{QK}^\top / \sqrt{d_k}\right) \mathbf{V}$$

$\mathbf{QK}^\top$ = ομοιότητες $\cdot \sqrt{d}$ σταθεροποιεί $\cdot \text{softmax} \rightarrow$ βάρη $\cdot$ σταθμισμένο άθροισμα των $\mathbf{V}$.



Προεκπαιδευμένοι Vision Encoders → Embeddings

- **Ο κοινός ρόλος**

- Εικόνα → διάνυσμα (embedding).
- CNN: global average pooling του feature map.
- Transformer: το [CLS] token.

- **Η εξέλιξη**

- AlexNet → VGG → Inception → ResNet (skip connections) → EfficientNet.

- **Τι κάνουμε με τα embeddings**

- Cosine similarity = οπτική ομοιότητα.
- Retrieval, clustering, k-NN, transfer learning.
- Προβολή σε 2-D (t-SNE, UMAP): φαίνεται αν οι κλάσεις ομαδοποιούνται χωρίς labels.
- Τα self-supervised μοντέλα δίνουν τα καθαρότερα clusters.

Μαθηματικά: Embeddings και Ομοιότητα

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

Γωνία δύο διανυσμάτων, ανεξάρτητη μέτρου. 1 = ίδια κατεύθυνση, 0 = ορθογώνια.

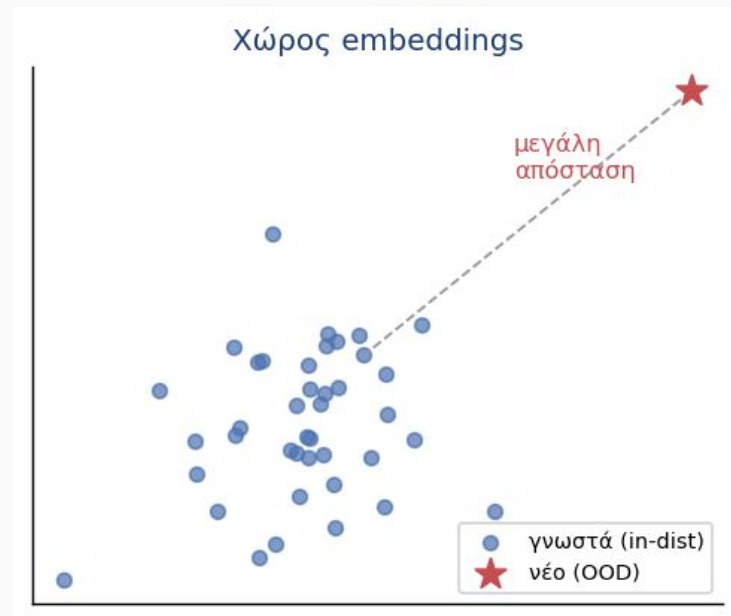


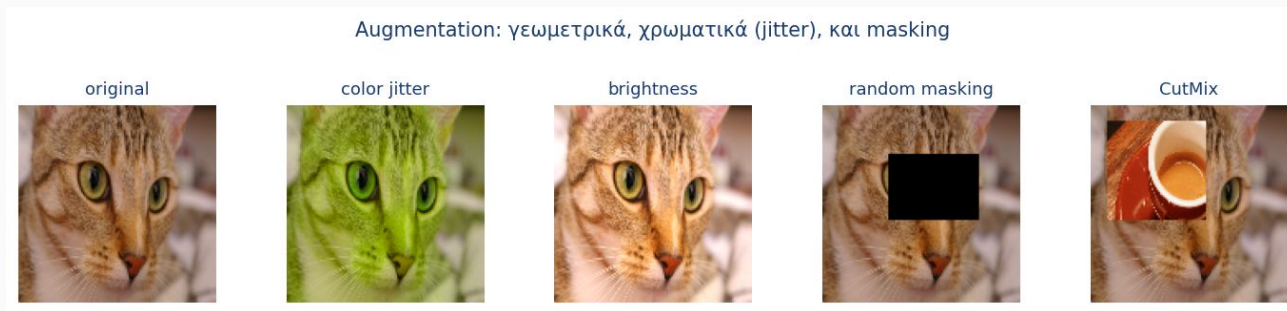
Image Augmentation

Στόχος: ποικιλία χωρίς νέα δεδομένα → λιγότερο overfitting

Γεωμετρικοί: flip, crop, rotation → αναλλοίωτο σε θέση/κλίμακα.

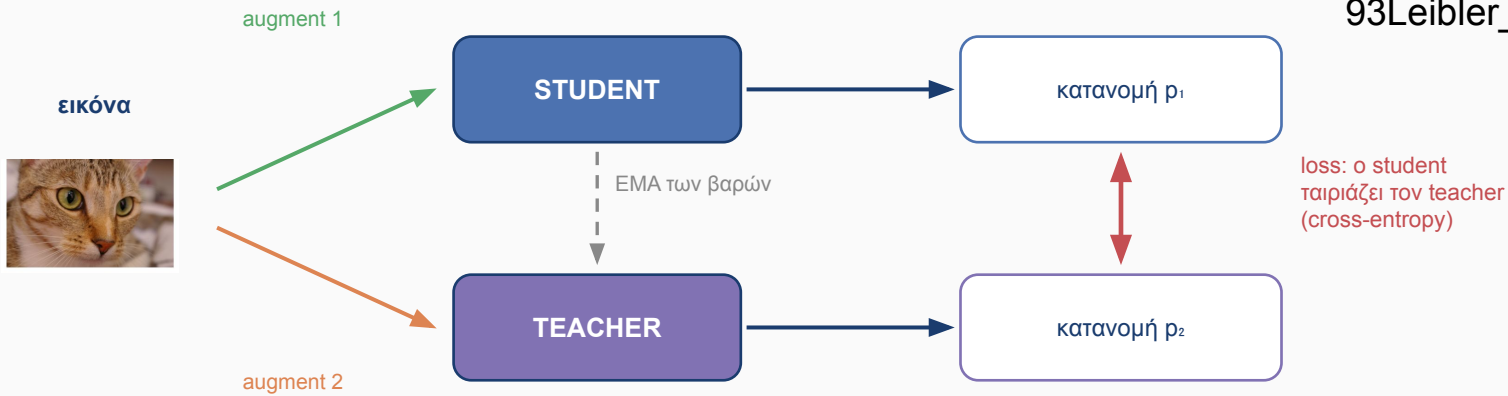
Color jitter: hue, saturation, brightness, contrast → αναλλοίωτο στον φωτισμό.

Masking / CutMix: κρύβει ή αντικαθιστά περιοχές → όχι εξάρτηση από ένα σημείο. Μόνο στην εκπαίδευση.



DINO: Self-Distillation w/o labels

Δύο augmentations → student & teacher · ο student μαθαίνει να ταιριάζει την έξοδο του teacher (cross-entropy).
Teacher = EMA των βαρών του student — δεν εκπαιδεύεται με gradient. Centering + sharpening αποτρέπουν την κατάρρευση.

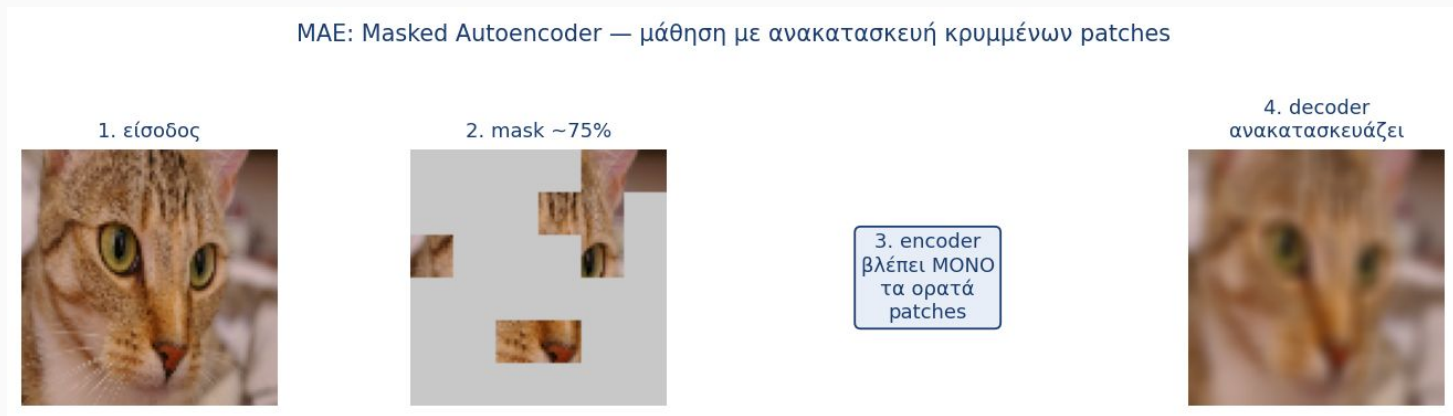


https://en.wikipedia.org/wiki/Kullback%E2%80%9393Leibler_divergence

MAE: Masked Autoencoder

Mask ~75% των patches · ο encoder βλέπει ΜΟΝΟ τα ορατά — φθινό.

Ελαφρύς decoder ανακατασκευάζει τα κρυμμένα pixels (MSE). Κρατάμε μόνο τον encoder — χαρακτηριστικά $\geq$ supervised, χωρίς annotation.

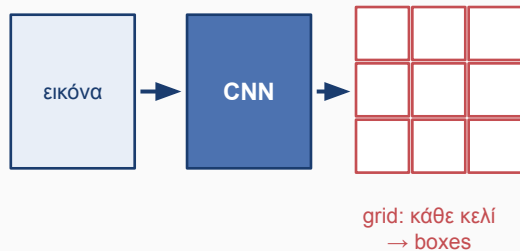


Object Detection: Τρεις Αρχιτεκτονικές

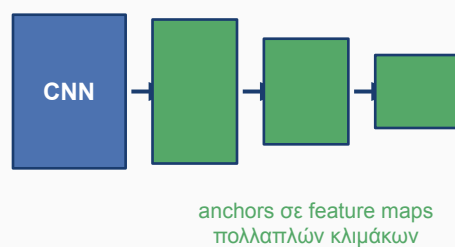
Στόχος: box + κλάση + confidence για κάθε αντικείμενο.

YOLO & SSD: one-stage με anchors — ταχύτητα · DETR: πρόβλεψη συνόλου με transformer — καθαρό pipeline.

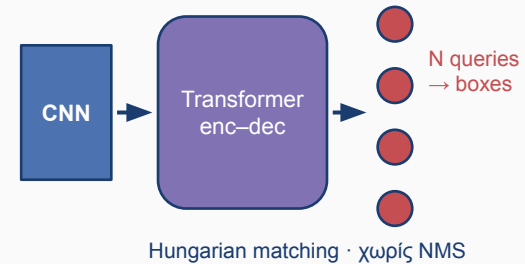
YOLO — one-stage, grid



SSD — multi-scale anchors



DETR — set prediction



YOLO — You Only Look Once

- **Πώς λειτουργεί**

- Grid πάνω στην εικόνα.
- Κάθε κελί προβλέπει boxes + κλάση σε ΕΝΑ πέρασμα.
- Το ταχύτερο → real-time (βίντεο, robotics).

- **Περιορισμοί**

- Μικρά και πυκνά αντικείμενα: λίγα boxes ανά κελί.
- Χαμηλότερη ακρίβεια εντοπισμού από βραδύτερα μοντέλα.
- Χρειάζεται anchors και NMS.

SSD – Single Shot Detector

- **Πώς λειτουργεί**

- One-stage με προκαθορισμένα anchors.
- Anchors σε feature maps πολλαπλών κλιμάκων
→ μεγέθη αντικειμένων.
- Καλή ισορροπία ταχύτητας–ακρίβειας.

- **Περιορισμοί**

- Μικρά αντικείμενα: εξαρτώνται από τη σχεδίαση των anchors.
- Χρειάζεται NMS.
- Πολλά αρνητικά anchors → class imbalance.

DETR — Detection Transformer

- **Πώς λειτουργεί**
 - CNN backbone → transformer encoder–decoder.
 - N learnable queries → πρόβλεψη ΣΥΝΟΛΟΥ boxes.
 - Hungarian matching · χωρίς anchors, χωρίς NMS.
- **Περιορισμοί**
 - Αργή σύγκλιση — πολλές epochs.
 - Αδύναμο σε πολύ μικρά αντικείμενα.
 - Attention $O(n^2)$ στην ανάλυση.

Μαθηματικά: IoU, Precision/Recall, mAP

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

Επικάλυψη box: σωστή πρόβλεψη αν $\text{IoU} \geq$ κατώφλι (π.χ. 0.5).

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}$$

Precision: σωστές / προβλέψεις · Recall: βρέθηκαν / πραγματικά.

$$\text{AP} = \int_0^1 P(R) dR, \quad \text{mAP} = \frac{1}{C} \sum_{c=1}^C \text{AP}_c$$

AP = εμβαδόν καμπύλης P-R · mAP = μέσος όρος στις C κλάσεις.

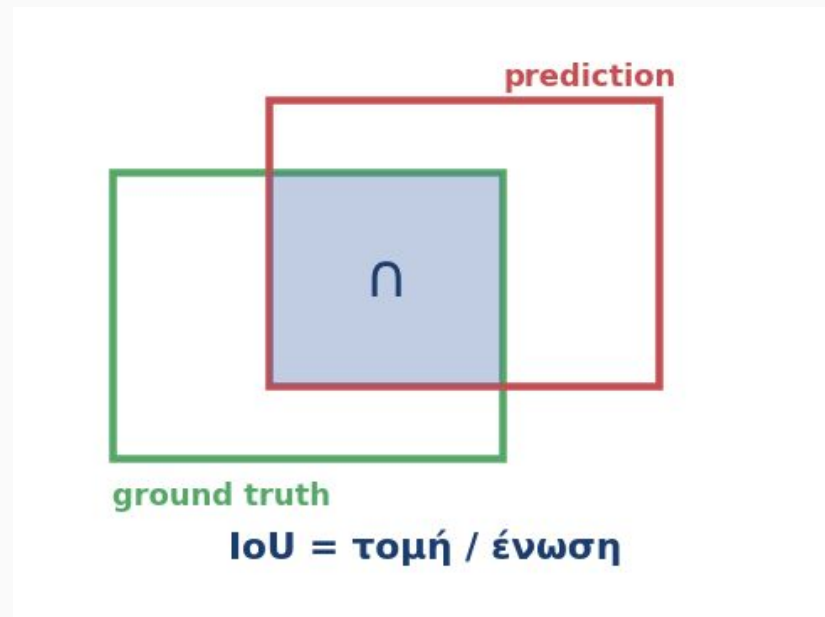
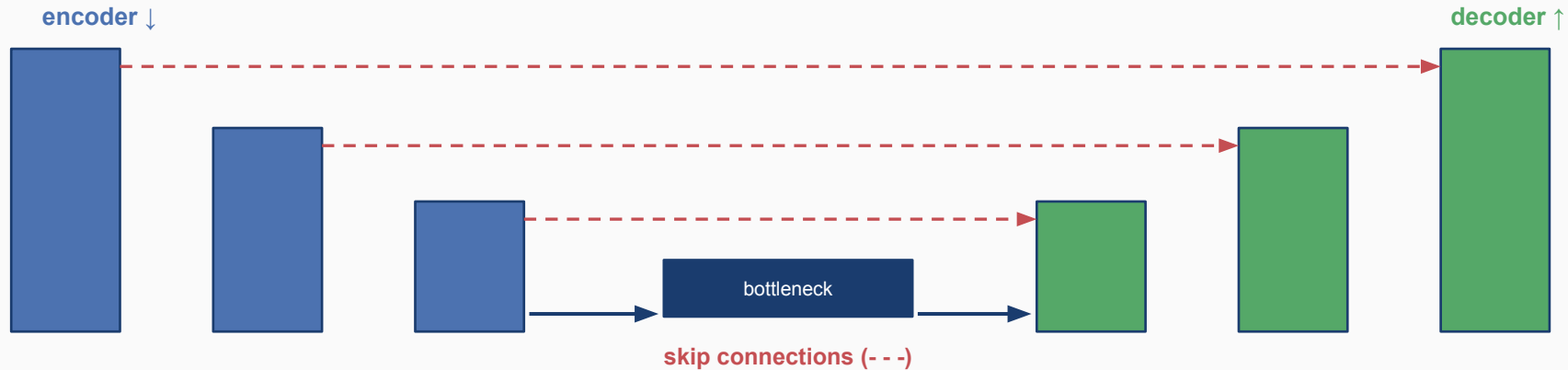


Image Segmentation (U-Net)

Μία ετικέτα ανά pixel: semantic (κλάση) · instance (αντικείμενο) · panoptic.

U-Net: encoder ↓ + decoder ↑ · skip connections μεταφέρουν λεπτομέρεια → ακριβή όρια.



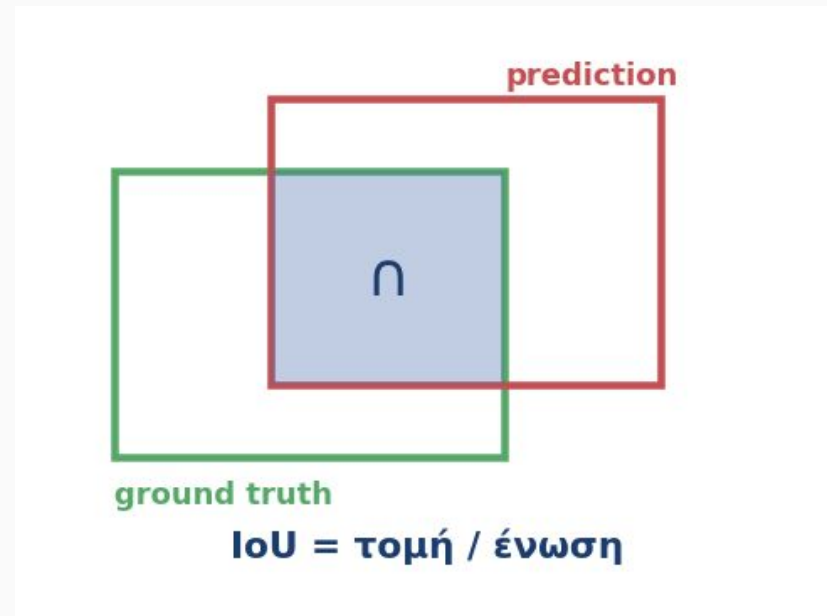
Μαθηματικά: Μετρική Segmentation (mIoU)

Ίδια δομή με το IoU της ανίχνευσης — ανά pixel αντί για κουτί.

$$\text{mIoU} = \frac{1}{C} \sum_{c=1}^C \frac{TP_c}{TP_c + FP_c + FN_c}$$

IoU ανά pixel για κάθε κλάση, μέσος όρος στις C.

TP/FP/FN μετρώνται σε pixels — όχι σε boxes.



Μαθηματικά: Loss του Segmentation

Στην πράξη συχνά συνδυάζονται: cross-entropy + Dice.

$$\mathcal{L}_{seg} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C g_{i,c} \log \hat{p}_{i,c}$$

Pixel-wise cross-entropy: CE σε ΚΑΘΕ pixel και κλάση — η βασική loss.

$$\mathcal{L}_{Dice} = 1 - \frac{2 \sum_i p_i g_i}{\sum_i p_i + \sum_i g_i}$$

Dice: μετρά απευθείας επικάλυψη — ανθεκτική σε class imbalance.

Καταμέτρηση: Deep vs Classical

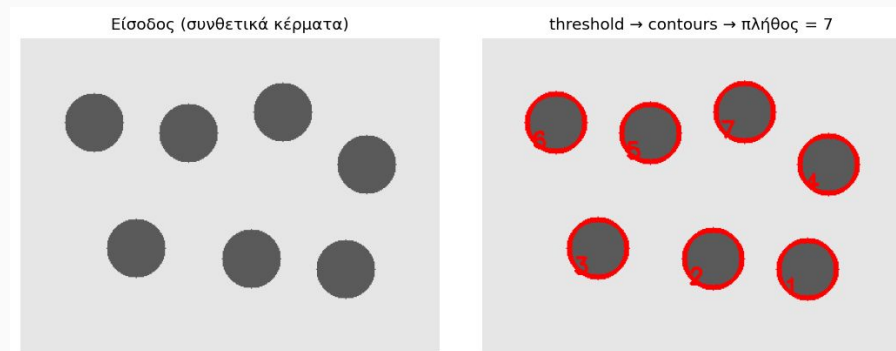
Με detector

Ανίχνευση → μέτρηση boxes ανά κλάση.
Για σύνθετες, ρεαλιστικές σκηνές.

Κλασικά — χωρίς μοντέλο

Threshold → contours → φιλτράρισμα area → πλήθος.
Σε ms, χωρίς GPU — για καθαρά, διαχωρισμένα blobs.

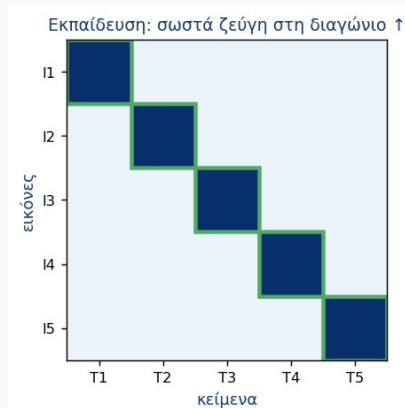
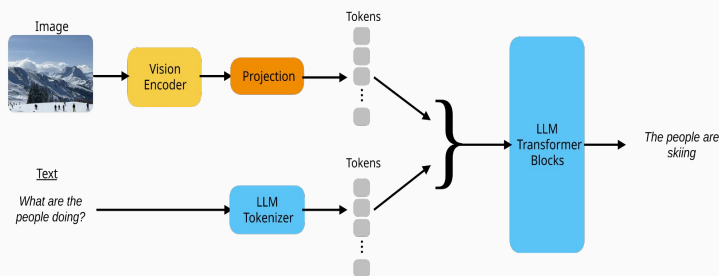
Γενικευμένος
Hough Transform
για το κλασικό CV



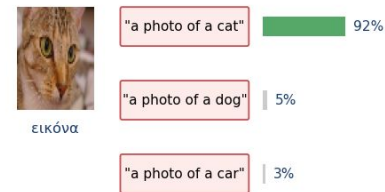
CLIP: Ταξινόμηση με Λέξεις (Zero-Shot)

Δύο encoders (εικόνας, κειμένου) → κοινός χώρος · contrastive εκπαίδευση σε 400M ζεύγη.

Zero-shot: κλάσεις ως προτάσεις → πλησιέστερη πρόταση = ετικέτα. Οι κλάσεις αλλάζουν χωρίς επανεκπαίδευση.



Zero-shot: κλάσεις ως προτάσεις



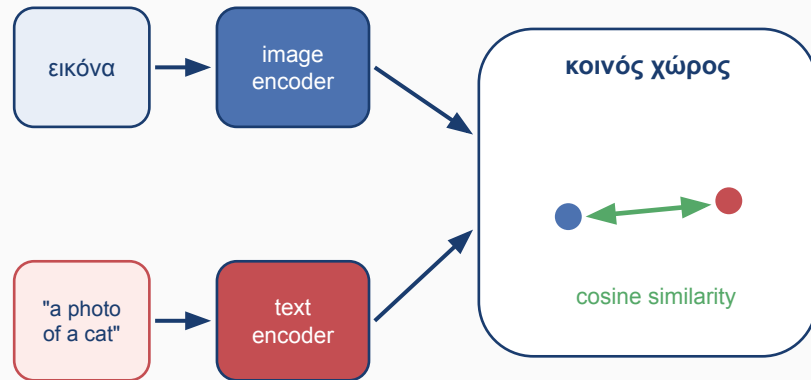
η εικόνα παίρνει την πλησιέστερη πρόταση (cosine similarity)

Μαθηματικά: Contrastive Learning (CLIP)

Εφαρμόζεται συμμετρικά: εικόνα→κείμενο και κείμενο→εικόνα.

$$\mathcal{L}_i = -\log \frac{\exp(\cos(\mathbf{u}_i, \mathbf{v}_i)/\tau)}{\sum_{j=1}^N \exp(\cos(\mathbf{u}_i, \mathbf{v}_j)/\tau)}$$

InfoNCE: το σωστό ζεύγος i πρέπει να νικά όλα τα άλλα N · η τ ρυθμίζει την «αιχμηρότητα».



Diffusion Models: Η Βασική Ιδέα

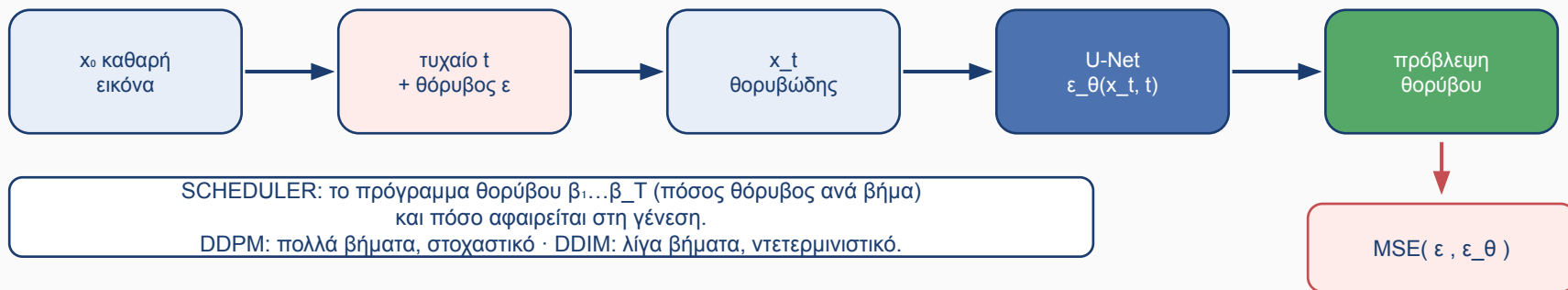
- **Forward (σταθερό)**
- Προσθήκη Gaussian θορύβου σε T βήματα $\rightarrow$ καθαρός θόρυβος.
- **Reverse (μαθαίνεται)**
- U-Net προβλέπει τον θόρυβο κάθε βήματος.
- Γένεση: από τυχαίο θόρυβο, αφαίρεση βήμα-βήμα.
- **Latent diffusion**
- Όλα στον latent χώρο ενός VAE ($\approx 8\times$ μικρότερος) $\rightarrow$ πολύ φθηνότερο.

Diffusion: Εκπαίδευση και Scheduler

Εκπαίδευση: τυχαίο t , πρόσθεσε θόρυβο ϵ , το ϵ_θ τον προβλέπει — μόνο MSE.

Scheduler: το πρόγραμμα θορύβου $\beta_1 \dots \beta_T$ και πόσο αφαιρείται ανά βήμα στη γένεση. DDPM: πολλά βήματα, στοχαστικό · DDIM: λίγα, ντετερμινιστικό.

Εκπαίδευση: πρόβλεψε τον θόρυβο που πρόσθεσες



Stable Diffusion: 4 Τρόποι Conditioning

- Conditioning = η καθοδήγηση της αποθορυβοποίησης. Ίδιο μοντέλο, ίδιο seed:
- TEXT — CLIP embedding μέσω cross-attention: «φαντάσου το».
- IMG2IMG — εκκίνηση από θορυβοποιημένη φωτογραφία: «ξαναζωγράφισέ το».
- INPAINT — αναγέννηση μόνο μέσα στο mask: «γέμισε την τρύπα».
- CONTROLNET — ακμές/πόζα/βάθος ως δομή: «ακολούθησε τις γραμμές».

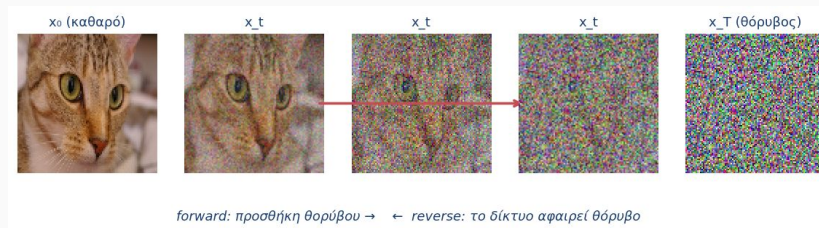
Μαθηματικά: Diffusion Models

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Reparameterization: το $\mathbf{x}_t$ απευθείας από $\mathbf{x}_0$ και $\boldsymbol{\varepsilon}$ — χωρίς προσομοίωση των ενδιάμεσων.

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0, \boldsymbol{\varepsilon}, t} \|\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_{\theta}(\mathbf{x}_t, t)\|^2$$

Στόχος: το $\boldsymbol{\varepsilon}_{\theta}$ προβλέπει τον θόρυβο — απλό MSE.



LoRA: Fine-Tuning

- **Η μέθοδος**

- Πάγωμα του W_0 · μάθηση μόνο $\Delta W = BA$ χαμηλού rank.
- Inference: $W' = W_0 + BA \rightarrow$ μηδενική καθυστέρηση.
- ~1% των βαρών · adapter λίγα MB.

- **Πού εφαρμόζεται**

- Ταξινομητές: προσαρμογή σε νέο dataset.
- Generators: νέο concept από λίγες εικόνες (DreamBooth-style).
- Ίδια συνταγή για κάθε αρχιτεκτονική.

Μαθηματικά: LoRA και Quantization

$$\mathbf{W}' = \mathbf{W}_0 + \Delta \mathbf{W} = \mathbf{W}_0 + \mathbf{B}\mathbf{A}, \quad r \ll d$$

$\Delta \mathbf{W} = \mathbf{B}\mathbf{A}$ με $\text{rank } r \ll d$ · εκπαιδεύονται μόνο B, A.

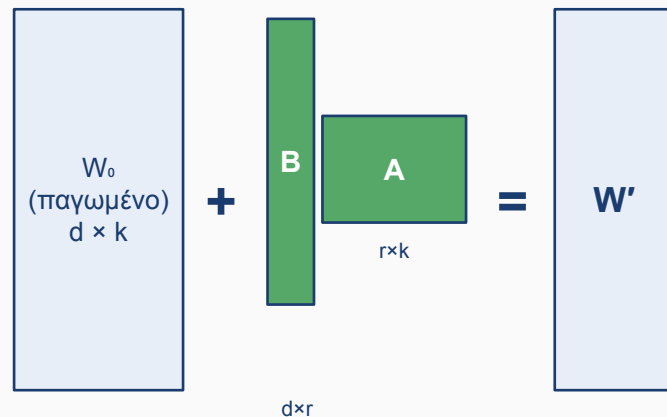
$$\# \text{trainable} / \# \text{total} = r(d + k) / (dk) \approx 1\%$$

Για $r = 8 \rightarrow \sim 1\%$ των βαρών.

$$x_{\text{int}} = \text{round}(x/s) + z, \quad s = (x_{\text{max}} - x_{\text{min}}) / 255$$

int8: κάθε βάρος $\rightarrow$ 8-bit ακέραιος με scale s , zero-point z .

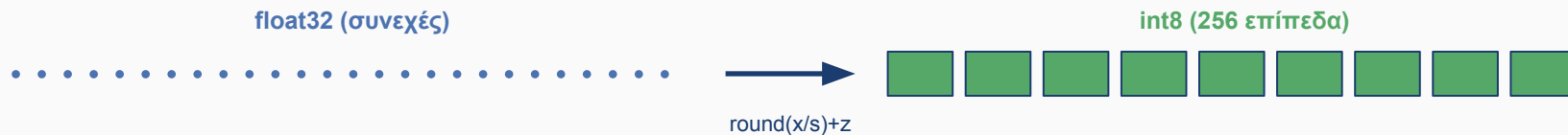
εκπαιδεύονται μόνο B, A ($\text{rank } r \ll d$) $\rightarrow \sim 1\%$ των βαρών



Quantization: float32 $\rightarrow$ int8

8 bits αντί 32 $\rightarrow$ 4× μικρότερο μοντέλο · int8 πράξεις ταχύτερες σε CPU.

Dynamic: κβάντιση των Linear layers, χωρίς επανεκπαίδευση. Μετράμε: μέγεθος, χρόνο/εικόνα, ίδια top-1; Τυπική απώλεια < 1%.



4× μικρότερο · ταχύτερο σε CPU · < 1% απώλεια ακρίβειας

Αξιόπιστη Όραση: OOD και Fake Detection

- **Out-of-Distribution**
- Ο ταξινομητής απαντά πάντα – και σε άγνωστη είσοδο.
- Score: 1 – μέγιστη cosine similarity προς γνωστά δείγματα.
- Υψηλό score = εκτός κατανομής.
- **Real vs AI**
- CLIP zero-shot: «πραγματική φωτογραφία» vs «εικόνα από AI».
- Ίδιο pattern: embed → σύγκριση αποστάσεων.
- Λύνει retrieval, matching, OOD, fake.

Μαθηματικά: OOD Score με Embeddings

Καμία εκπαίδευση — μόνο embeddings και αποστάσεις.

$$s_{OOD}(\mathbf{x}) = 1 - \max_{q \neq x} \cos(f(\mathbf{x}), f(\mathbf{q}))$$

1 – μέγιστη ομοιότητα του $f(\mathbf{x})$ προς γνωστό δείγμα · χαμηλή ομοιότητα → υψηλό score.

